LIBRARY MANAGEMENT SYSTEM USING SPRING BOOT REST API

A SUMMER TRAINING INTERNSHIP REPORT

Submitted by

Himanshi (UID: 24BCS12349)
Shubham Jindal (UID: 24BCS10832)

in partial fulfilment of the requirements of the In-House Summer Training Internship

towards the award of the degree of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING (BE-CSE)

Under the Guidance of
Ankita Sharma, Assistant Professor

INSTITUTE – UNIVERSITY INSTITUTE OF ENGINEERING (UIE)

CHANDIGARH UNIVERSITY

July 2026

CHANDIGARH UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this Summer Training Internship report titled "LIBRARY MANAGEMENT SYSTEM USING SPRING BOOT REST API" is the bonafide work of Himanshi (UID: 24BCS12349) and Shubham Jindal (UID: 24BCS10832), who carried out the In-House Summer Training Internship under my/our supervision during June–July 2026, academic session 2025–2026.

HoD Signature	Supervisor Signature
Dr. Jaspreet Singh Batth	**Ankita Sharma**
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Department of Computer Science and Engineering	Assistant Professor, Department of Computer Science and Engineering

Submitted for the Summer Training Internship viva-voce examination held on _____.

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to Chandigarh University for providing us with the platform and resources to undertake our In-House Summer Training Internship on "Library Management System using Spring Boot REST API", carried out as part of the curriculum of the Department of Computer Science and Engineering.

We are deeply thankful to our internship supervisor, Ankita Sharma, Assistant Professor, Department of Computer Science and Engineering, for her invaluable guidance, constant encouragement, and constructive suggestions throughout the duration of the Summer Training. Her support was instrumental in helping us understand the practical aspects of backend web development and in successfully completing this internship work.

We would also like to thank Dr. Jaspreet Singh Batth, Head of the Department, and all the faculty members of the Department of Computer Science and Engineering for creating an environment conducive to learning and for equipping us with the theoretical and practical knowledge required to design and develop this project.

Finally, we extend our heartfelt thanks to our families and friends for their continuous motivation and support during the course of this Summer Training Internship.

Himanshi (UID: 24BCS12349)

Shubham Jindal (UID: 24BCS10832)

ABSTRACT

Efficient management of a library's book records is a fundamental requirement for any educational institution. Manual record-keeping methods are increasingly being replaced by software-driven solutions that are faster, more accurate, and easier to maintain. This report presents the work carried out during an In-House Summer Training Internship at Chandigarh University, in which a backend RESTful web service, "Library Management System using Spring Boot REST API," was designed and developed to allow a library's book inventory to be created, retrieved, updated, and deleted through a standard set of HTTP-based Application Programming Interface (API) endpoints.

The system was built using Java as the programming language and the Spring Boot framework, which simplifies the configuration and bootstrapping of Java web applications. Persistence of book data is handled through Spring Data JPA in conjunction with Hibernate as the Object-Relational Mapping (ORM) provider, allowing the Java Book entity to be automatically mapped onto a relational table in an MySQL 8.0 Relational Database instance. The application follows a layered architecture consisting of a Controller layer that exposes REST endpoints, a Service layer that encapsulates business logic, and a Repository layer that interacts with the underlying database through Spring Data JPA's JpaRepository interface. The project was built and dependency-managed using Apache Maven and developed within the Eclipse Integrated Development Environment (IDE).

Five core REST endpoints were implemented and tested during the internship: adding a new book record (HTTP POST), retrieving all book records (HTTP GET), retrieving a specific book record by its identifier (HTTP GET), updating an existing book record (HTTP PUT), and deleting a book record (HTTP DELETE). Each endpoint was rigorously tested using Postman, and the corresponding changes were independently verified by querying the MySQL database directly through MySQL Workbench. The Spring Boot console logs further confirmed that the embedded Tomcat server, Hibernate ORM, and the HikariCP connection pool were functioning correctly and that the application successfully connected to the MySQL database at start-up.

The successful implementation and testing of all Create, Read, Update, and Delete (CRUD) operations demonstrate that the system developed during this Summer Training Internship offers a reliable, extensible, and maintainable foundation for a library management

solution. This report documents the complete life cycle of the internship project, from problem identification and system design through implementation, testing, and results, and concludes with a discussion of the system's advantages, limitations, and possible future enhancements such as authentication, book issue/return tracking, and pagination.

Keywords: *Spring Boot, REST API, Hibernate, Java Persistence API (JPA), MySQL Database, CRUD Operations, Postman, Maven, Library Management System.*

ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
REST	Representational State Transfer
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
CRUD	Create, Read, Update, Delete
ORM	Object-Relational Mapping
JPA	Java Persistence API
JDBC	Java Database Connectivity
IDE	Integrated Development Environment
XE	Express Edition (MySQL Database)
SQL	Structured Query Language
POM	Project Object Model
JWT	JSON Web Token
UID	University Identification (Student ID)

TABLE OF CONTENTS

Abstract 4
Abbreviations 6
List of Figures 8
List of Tables 9
Chapter 1: Introduction 10
Chapter 2: Literature Review 15
Chapter 3: System Requirements 18
Chapter 4: System Design 20
Chapter 5: Database Design 24
Chapter 6: Project Implementation 27
Chapter 7: REST API Implementation 35
Chapter 8: Testing 37
Chapter 9: Results 45
Chapter 10: Advantages 47
Chapter 11: Limitations 49
Chapter 12: Future Enhancements 51
Conclusion 53
References 54

LIST OF FIGURES

Figure 4.1 Layered Architecture of the Library Management System

Figure 4.2 Request Flow for a POST /books Request

Figure 5.1 library_book Table Structure in MySQL Workbench

Figure 6.1 Project Structure in Eclipse (Expanded View)

Figure 6.2 Project Structure in Eclipse (Collapsed View)

Figure 6.3 Spring Boot Console Output During Application Start-up

Figure 7.1 Postman Collection Showing All Implemented REST API Endpoints

Figure 8.1 POST /books — Add Book Request and Response

Figure 8.2 GET /books — Retrieve All Books Request and Response

Figure 8.3 GET /books/{id} — Retrieve Book by ID Request and Response

Figure 8.4 PUT /books — Update Book Request and Response

Figure 8.5 DELETE /books/{id} — Delete Book Request and Response

Figure 8.6 Oracle SQL Verification — After POST and PUT Operations

Figure 8.7 Oracle SQL Verification — Re-Query Confirming Persistence

Figure 8.8 Oracle SQL Verification — After the DELETE Operation

LIST OF TABLES

Table 3.1 Hardware Requirements

Table 3.2 Software Requirements

Table 5.1 Column Description of the library_book Table

Table 5.2 Mapping Between the Book Entity and the library_book Table

Table 6.1 Package Structure of the Library Management Application

Table 6.2 Key Properties Configured in application.properties

Table 6.3 Key Maven Dependencies Declared in pom.xml

Table 7.1 REST API Endpoint Summary for the Library Management System

CHAPTER 1
INTRODUCTION

1.1 About the Summer Training Internship

This report documents the work carried out by the authors during an In-House Summer Training Internship at the University Institute of Engineering (UIE), Chandigarh University, conducted over a period of four weeks between June and July 2026. The internship required each participant to design, implement, and test a functional backend software system, applying concepts of object-oriented programming, web application frameworks, and relational database management learned during the undergraduate curriculum to a practical, self-contained project.

For this internship, the authors developed a Library Management System REST API, a backend web service that allows a library's book inventory to be created, retrieved, updated, and deleted through a set of HTTP-based endpoints. The following sections introduce the problem addressed by the internship project, the technologies used, and the objectives and scope of the work completed.

1.2 Introduction to the Project

Libraries, whether in schools, colleges, or universities, are responsible for maintaining accurate and up-to-date records of the books available in their collection. Traditionally, this record-keeping was performed manually using registers or, later, using simple spreadsheet-based systems. As the number of books and the frequency of transactions grow, manual and spreadsheet-based approaches become increasingly difficult to maintain, are prone to human error, and do not scale well when multiple users or applications need to access the same data concurrently.

This internship project, titled “Library Management System using Spring Boot REST API,” addresses this problem by developing a backend web service that exposes a set of RESTful Application Programming Interfaces (APIs) for managing book records. The system was developed using Java 17 (project language level) executed on a JDK 21 runtime within the VS Code / IDE, the Spring Boot 3 framework, Spring Data JPA with Hibernate as the ORM provider, and MySQL 8.0 Relational Database as the persistent data store. Apache Maven was used to manage project dependencies and the build lifecycle, and Postman was used extensively to test and validate every API endpoint.

Rather than being a monolithic application with a built-in user interface, the system was deliberately designed as a backend REST service. By exposing the business logic and data access functionality purely through HTTP-based REST endpoints, the same backend can, in principle, be consumed by multiple types of client applications — a web front-end, a mobile application, or a third-party integration — without requiring any change to the underlying service.

1.3 Problem Statement

Educational institutions and small libraries often continue to rely on manual registers or generic spreadsheet files to record details of the books they hold, such as the title, author, category, and price of each book. This approach suffers from several practical problems: records can be misplaced or duplicated, simultaneous updates by multiple staff members can lead to inconsistent data, there is no structured way to query the collection, and there is no mechanism to expose this data programmatically to other applications. There is, therefore, a need for a lightweight, structured, and programmatically accessible system that can reliably store and manage book records and make them available through a well-defined interface.

1.4 Existing System

In many small institutions, the existing system for managing library book records is one of the following:

- A manual register in which books are recorded and updated by hand.
- A spreadsheet file (for example, an Excel workbook) maintained by library staff, updated manually whenever a book is added, its details change, or it is removed from the collection.
- In slightly more advanced setups, a desktop application with a local file-based store that is not accessible outside the machine on which it runs and offers no standard programmatic interface.

1.5 Limitations of the Existing System

The existing manual or spreadsheet-based systems described above suffer from the following limitations:

- No centralized, structured database: data is often stored in flat files or paper registers with no relational structure or referential integrity.
- High risk of data redundancy and inconsistency: the same book record may be entered more than once, or updates made by one user may not be reflected in copies held by others.
- No programmatic access: there is no standard way for another application to read or update the records.
- Poor scalability: as the number of books grows, manual systems become slow and error-prone to search and update.
- No validation or automation: there is no automatic enforcement of data types, constraints, or business rules.
- Limited concurrent access: multiple users cannot reliably work with the same data set at the same time without the risk of conflicting updates.

1.6 Proposed System

The proposed system replaces manual or spreadsheet-based record keeping with a Spring Boot based REST API backed by an Oracle relational database. Book records (identifier, title, author, price, and category) are persisted in a dedicated library_book table, and all operations on this data — creating, reading, updating, and deleting book records — are exposed through well-defined HTTP endpoints under the /books resource path. The system follows a layered architecture (Controller, Service, Repository) that cleanly separates the concerns of request handling, business logic, and data access, in line with standard Spring Boot design conventions. Hibernate, operating beneath Spring Data JPA, automatically maps the Java Book entity class to the corresponding table in the MySQL database, removing the need for hand-written SQL for routine CRUD operations.

1.7 Objectives of the Internship Project

The objectives defined for this Summer Training Internship project are as follows:

- To design and implement a RESTful backend service for managing library book records using Spring Boot.
- To model the book record as a Java entity class and persist it in an MySQL Database using Spring Data JPA and Hibernate ORM.

- To implement and expose REST endpoints supporting all four CRUD operations — Create, Read, Update, and Delete — on book records.
- To structure the application using a layered architecture (Controller, Service, Repository) that promotes separation of concerns and maintainability.
- To verify the correctness of every implemented endpoint using Postman as an independent API testing tool.
- To independently confirm that changes made through the REST API are correctly persisted by directly querying the MySQL database using MySQL Workbench.
- To document the complete design, implementation, and testing process in a manner suitable for academic evaluation.

1.8 Scope

The scope of the current implementation covers the management of book records only, through the Book entity, which stores the identifier, title, author, price, and category of each book. The system supports the four fundamental CRUD operations on this entity through REST endpoints, and all operations were tested using Postman with results verified in MySQL Workbench. The scope of this internship project, in its current form, does not extend to user authentication, member/borrower management, book issue and return tracking, or a front-end user interface; these are identified as potential directions for future enhancement in Chapter 12 of this report.

1.9 Advantages

The proposed system, even in its current scope, offers the following advantages over the existing manual approach:

- Centralized and structured storage of book records in a relational MySQL database.
- Elimination of duplicate or manually re-entered records through a single, unique identifier per book.
- Standardized, technology-agnostic access to book data through REST endpoints that can be consumed by any HTTP client.
- Clear separation of concerns through a layered (Controller–Service–Repository) architecture, improving maintainability and testability.

- Reduced boilerplate database code through Spring Data JPA's JpaRepository, which provides ready-made CRUD methods.
- Automatic schema management during development through Hibernate's ddl-auto=update configuration.
- Ease of testing through tools such as Postman, without requiring a graphical front-end to be built first.

CHAPTER 2
LITERATURE REVIEW

This chapter briefly reviews the core technologies and concepts that underpin the design and implementation of the Library Management System REST API developed during the Summer Training Internship. Each of the following topics maps directly to a technology used in the project, as listed in Chapter 3 of this report.

2.1 REST API (Representational State Transfer)

Representational State Transfer (REST) is an architectural style for designing networked applications, most commonly implemented over the HTTP protocol. In a REST-based system, resources (in this project, book records) are identified by URIs, and operations on those resources are expressed using standard HTTP methods: POST to create a resource, GET to read a resource, PUT to update a resource, and DELETE to remove a resource. REST APIs are stateless, meaning each request from a client contains all the information the server needs to process it, and responses are typically exchanged in a lightweight, human-readable format such as JSON. This project implements a REST API for the /books resource, mapping each CRUD operation to the corresponding HTTP method, as demonstrated in Chapter 7 of this report.

2.2 Spring Boot

Spring Boot is an extension of the Spring Framework that simplifies the development of stand-alone, production-ready Spring applications by minimizing configuration overhead. It provides an embedded web server (Apache Tomcat, in this project), auto-configuration based on the dependencies present on the classpath, and a simple entry point (a class annotated with @SpringBootApplication) from which the entire application can be started. In this project, Spring Boot 3.5.16 is used as the foundation of the backend service, providing the embedded Tomcat server that listens on port 8082, dependency injection through annotations such as @Autowired, and the @RestController and mapping annotations (@GetMapping, @PostMapping, @PutMapping, @DeleteMapping) that are used to expose REST endpoints.

2.3 Hibernate ORM

Hibernate is an Object-Relational Mapping (ORM) framework for Java that maps Java objects (entities) to rows in relational database tables, removing the need to write repetitive

SQL statements for common data access operations. Hibernate manages the translation between the object-oriented world of Java classes and the tabular structure of a relational database, handling tasks such as generating SQL statements, managing database connections, and tracking changes to persisted objects. In this project, Hibernate ORM (version 6.6.53.Final, as confirmed in the Spring Boot console output) is used beneath Spring Data JPA to map the Book entity class to the library_book table in the MySQL database.

2.4 MySQL Database

MySQL Database is a widely used commercial relational database management system (RDBMS) that supports structured storage of data in tables, along with standard relational database features such as primary keys, data types, and SQL-based querying. This project uses MySQL 8.0 Relational Database, a free, lightweight edition of MySQL Database suitable for development and educational use, as confirmed by the database version 8.0 reported in the Spring Boot console output and the JDBC connection string used in the application configuration (jdbc:mysql://localhost:3306/library_management). The book records created through the REST API are persisted in a single table, library_book, within this Oracle instance.

2.5 Java Persistence API (JPA)

The Java Persistence API (JPA) is a specification that defines a standard way to manage relational data in Java applications, including annotations such as @Entity, @Id, and @Table used to describe how a Java class maps to a database table. Spring Data JPA is a Spring project that builds on top of JPA (implemented, in this project, by Hibernate) and further simplifies data access by providing the JpaRepository interface, which automatically supplies implementations for common operations such as save(), findAll(), findById(), and deleteById() without requiring the developer to write any implementation code. This project's BookRepository interface extends JpaRepository<Book, Integer> to obtain these operations directly.

2.6 Apache Maven

Apache Maven is a build automation and dependency management tool for Java projects. It uses a project object model, defined in a pom.xml file, to describe the project's dependencies, plugins, and build configuration. Maven automatically downloads the

required libraries (such as the Spring Boot starters, the Oracle JDBC driver, and Lombok) from a central repository and manages the project's build lifecycle, including compiling source code, running tests, and packaging the application. In this project, Maven (using the Maven Wrapper, mvnw) is used to manage all project dependencies, as declared in the project's pom.xml file, which is discussed in detail in Chapter 6.

CHAPTER 3
SYSTEM REQUIREMENTS

This chapter outlines the hardware and software requirements, as well as the development tools, that were used to build, run, and test the Library Management System REST API during the Summer Training Internship.

3.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 (or equivalent) or higher
RAM	4 GB minimum (8 GB recommended)
Hard Disk Space	10 GB free space (for JDK, IDE, MySQL DB, and project files)
Operating System	Windows 10 / 11 (64-bit)
Network	Local network / localhost access for application–database communication

Table 3.1: Hardware Requirements

3.2 Software Requirements

Software	Version / Details
Operating System	Windows
Programming Language	Java (JavaSE-17 project configuration, executed on JDK 21.0.11 runtime)
Framework	Spring Boot 3.5.16
Persistence Framework	Spring Data JPA with Hibernate ORM 6.6.53.Final
Database	MySQL Community Server 8.0
Build Tool	Apache Maven (via Maven Wrapper — mvnw)
API Testing Tool	Postman
Database Client	MySQL Workbench 8.0

Table 3.2: Software Requirements

3.3 Development Tools

The following development tools were used throughout the design, implementation, and testing of this internship project:

- VS Code / IDE — used to write, organize, compile, and run the Spring Boot application, and to monitor the application's console output during start-up and execution.
- Apache Maven (mvnw / mvnw.cmd) — used to manage project dependencies declared in pom.xml and to build the project.
- MySQL Workbench — used to connect to the MySQL 8.0 Database instance, inspect the library_book table structure, and independently verify that data created, updated, and deleted through the REST API was correctly reflected in the database.
- Postman — used to construct and send HTTP requests (POST, GET, PUT, DELETE) to the running Spring Boot application and inspect the JSON responses returned by each endpoint.
- Spring Boot embedded Apache Tomcat server — used to host the REST API on port 8082 without requiring a separately installed application server.

CHAPTER 4
SYSTEM DESIGN

4.1 Architecture Overview

The Library Management System REST API follows a standard three-layer (Controller–Service–Repository) architecture, the conventional design pattern recommended for Spring Boot applications. This layered approach separates the concerns of handling HTTP requests, implementing business logic, and accessing the database, making the application easier to understand, test, and extend. Each layer in the current project communicates only with the layer immediately below it: the Controller layer depends on the Service layer, and the Service layer depends on the Repository layer, which in turn relies on Hibernate and Spring Data JPA to communicate with the MySQL database.

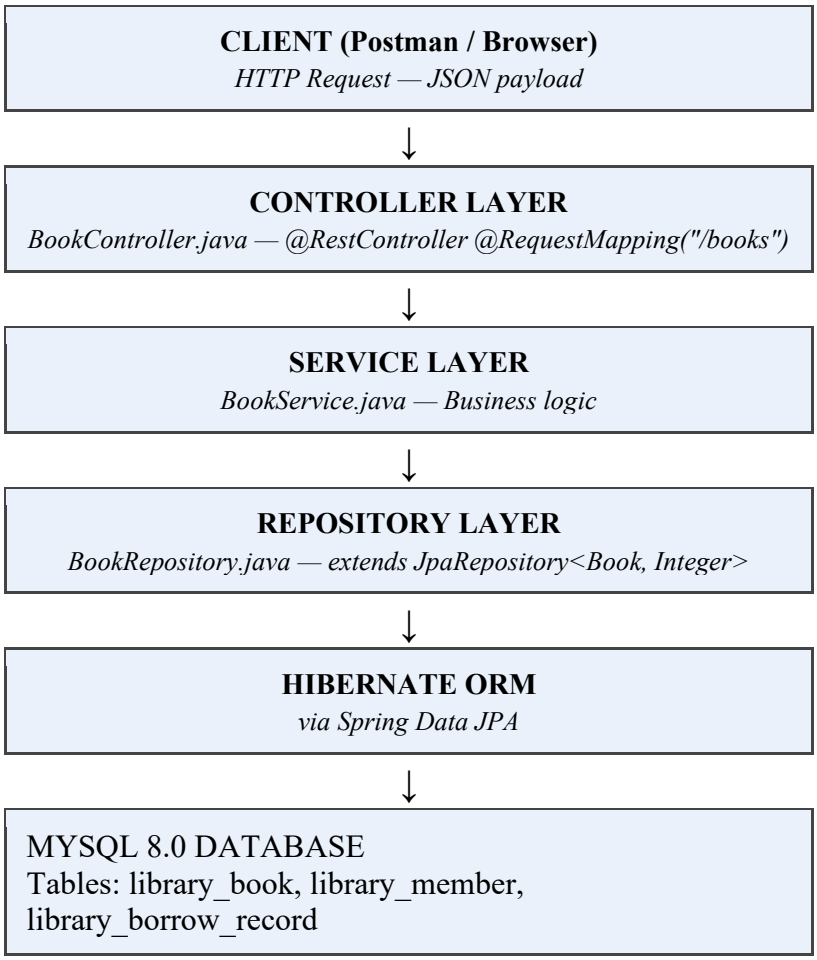


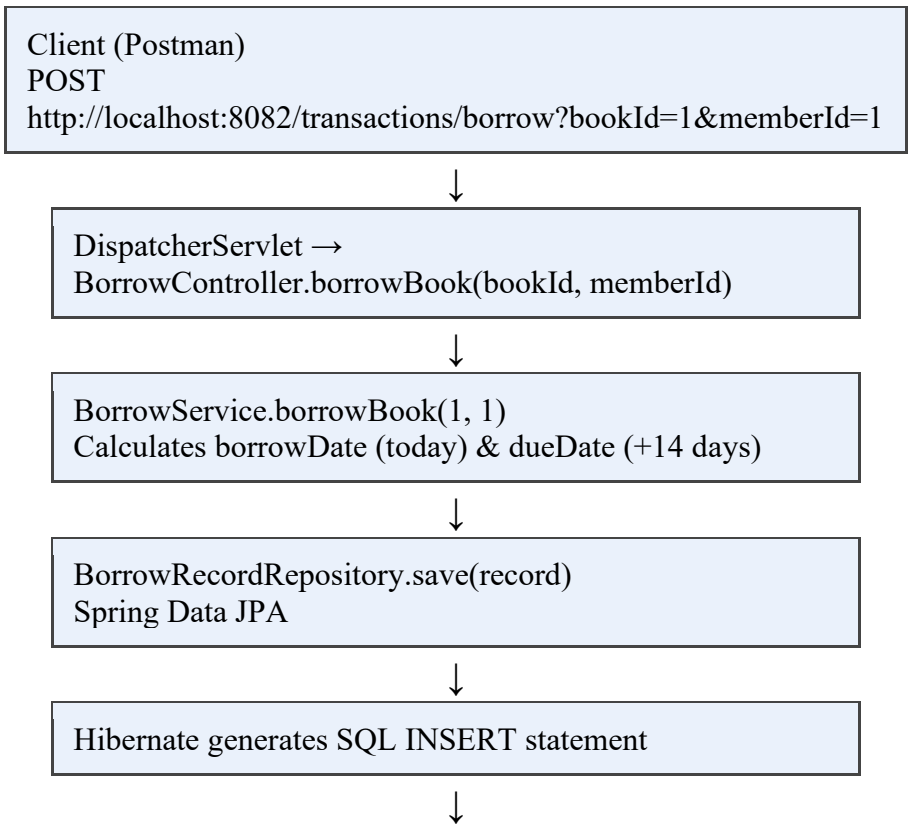
Figure 4.1: Layered Architecture of the Library Management System

4.2 Workflow

The overall workflow of the system begins when a client application — in this project, Postman — sends an HTTP request to one of the endpoints exposed under the /books path on port 8082. Spring Boot's embedded Tomcat server receives the request and routes it, through the Spring DispatcherServlet, to the appropriate method in BookController based on the HTTP method and URL path. The controller method delegates the request to the corresponding method in BookService, which in turn calls the appropriate method on BookRepository. Because BookRepository extends Spring Data JPA's JpaRepository interface, calls such as save(), findAll(), findById(), and deleteById() are automatically implemented, with Hibernate translating each call into the corresponding SQL statement executed against the MySQL database. The result of the database operation is passed back up through the Repository and Service layers to the Controller, which serializes it into a JSON response and returns it to the client.

4.3 Request Flow — Example: Adding a New Book (POST)

To illustrate the request flow concretely, Figure 4.2 traces the path taken by a single POST request that adds a new book to the system, from the moment Postman sends the request to the moment the newly created Book object is returned as a JSON response.



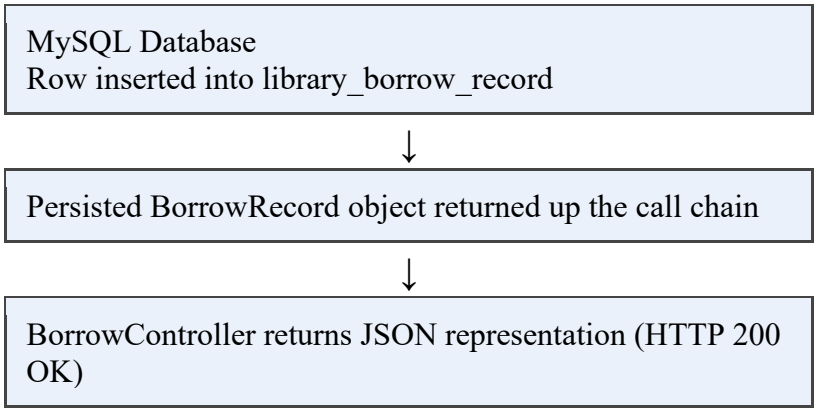


Figure 4.2: Request Flow for a POST /books Request

4.4 Controller Layer

The Controller layer is implemented in the BookController class, annotated with @RestController and @RequestMapping("/books"). This layer is responsible only for receiving HTTP requests, extracting data from the request (such as a JSON request body or a path variable), invoking the appropriate method on the Service layer, and returning the result, which Spring Boot automatically serializes to JSON. The Controller layer does not itself contain any business logic or direct database access code; its sole responsibility is to act as the entry point for HTTP requests and to translate between the HTTP world and the application's internal service methods.

4.5 Service Layer

The Service layer is implemented in the BookService class, annotated with @Service, and is responsible for encapsulating the business logic of the application. In the current implementation, the Service layer methods (saveBook, getAllBooks, getBookById, updateBook, and deleteBook) primarily delegate to the corresponding Repository methods, while also implementing simple logic such as checking whether a book exists before attempting to delete it. Placing this logic in a dedicated Service layer, rather than directly in the Controller, keeps the business rules separate from the HTTP-handling code and makes it straightforward to extend the business logic in the future without modifying the Controller.

4.6 Repository Layer

The Repository layer is implemented in the BookRepository interface, which extends Spring Data JPA's JpaRepository<Book, Integer>. By extending this interface,

BookRepository automatically inherits a full set of CRUD methods — including save(), findAll(), findById(), existsById(), and deleteById() — without requiring any method bodies to be written. Spring Data JPA generates the implementation of this interface at runtime, and Hibernate translates each method call into the appropriate SQL statement to be executed against the MySQL database.

4.7 Database Layer

The Database layer consists of the MySQL 8.0 Database instance running on localhost, port 3306, and specifically the library_book table within it, which stores all book records created through the REST API. The connection between the Spring Boot application and the MySQL database is managed by the HikariCP connection pool (visible in the Spring Boot console output as HikariPool-1), which is Spring Boot's default and recommended connection pooling implementation. The database layer is discussed in full detail, including the table schema and the mapping between the Java entity and the database table, in Chapter 5.

CHAPTER 5
DATABASE DESIGN

5.1 Overview

The Library Management System uses a single MySQL Database table, library_book, to store all book records. This table was generated automatically by Hibernate from the Java Book entity class, because the application property spring.jpa.hibernate.ddl-auto is set to update (see application.properties, discussed in Chapter 6), which instructs Hibernate to automatically create or update the database schema to match the entity definitions at application start-up. This chapter documents the structure of this table exactly as observed in MySQL Workbench, and explains how it relates to the corresponding Java entity class.

5.2 library_book Table Structure

Figure 5.1 shows the column structure of the library_book table as displayed in MySQL Workbench, obtained by describing the table after the application had been run and the schema generated by Hibernate.



Figure 5.1: library_book Table Structure in MySQL Workbench

5.3 Column Description

Table 5.1 explains the purpose and data type of every column present in the library_book table, based directly on the schema shown in Figure 5.1.

Column	Data Type	Nullable	Description
id	INT	No	Primary key of library_book. Uniquely identifies each book. Mapped from Book.id.
title	VARCHAR(255)	Yes	Stores the title of the book. Mapped from Book.title.

Column	Data Type	Nullable	Description
author	VARCHAR(255)	Yes	Stores the author of the book. Mapped from Book.author.
price	DOUBLE	No	Stores the price of the book. Mapped from Book.price.
category	VARCHAR(255)	Yes	Genre or category of the book (e.g. Programming). Mapped from Book.category.
member.id	BIGINT AUTO_INCREMENT	No	Primary key of library_member. Auto-generated identifier for members.
member.name	VARCHAR(255)	No	Full name of the registered library member.
member.email	VARCHAR(255)	Yes	Email address of the library member for notifications.
member.phone	VARCHAR(255)	Yes	Contact phone number of the library member.
borrow.id	BIGINT AUTO_INCREMENT	No	Primary key of library_borrow_record. Unique loan transaction ID.
borrow.book_id	INT	No	Foreign key referencing library_book(id).
borrow.member_id	BIGINT	No	Foreign key referencing library_member(id).
borrow.borrow_date	DATE	No	Date the book was issued to the member.
borrow.due_date	DATE	No	Date the book is due back (calculated as borrow_date + 14 days).
borrow.return_date	DATE	Yes	Date the book was returned (null while currently on loan).

Table 5.1: Column Description of the library_book Table

5.4 Equivalent SQL CREATE TABLE Statement

Although the library_book table was generated automatically by Hibernate rather than through a hand-written script, the equivalent SQL CREATE TABLE statement — derived directly from the column structure shown in Figure 5.1 — is given below for reference and documentation purposes.

```
-- MySQL 8.0 DDL for Library Management System
CREATE TABLE library_book (
    id INT NOT NULL,
    title VARCHAR(255),
    author VARCHAR(255),
    price DOUBLE NOT NULL,
```

```
    category VARCHAR(255),
    PRIMARY KEY (id)
);

CREATE TABLE library_member (
    id BIGINT AUTO_INCREMENT NOT NULL,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255),
    phone VARCHAR(255),
    PRIMARY KEY (id)
);

CREATE TABLE library_borrow_record (
    id BIGINT AUTO_INCREMENT NOT NULL,
    book_id INT NOT NULL,
    member_id BIGINT NOT NULL,
    borrow_date DATE NOT NULL,
    due_date DATE NOT NULL,
    return_date DATE,
    PRIMARY KEY (id),
    CONSTRAINT fk_borrow_book FOREIGN KEY (book_id) REFERENCES
library_book(id),
    CONSTRAINT fk_borrow_member FOREIGN KEY (member_id) REFERENCES
library_member(id)
);
```

5.5 Entity–Table Relationship

The mapping between the Java Book entity class and the library_book Oracle table is established declaratively using JPA annotations, without any hand-written SQL or XML mapping files. The @Entity annotation marks the Book class as a JPA-managed entity, and @Table(name = "library_book") explicitly specifies that this entity should be mapped to a table named library_book. Within the entity, the id field is annotated with @Id, designating it as the primary key column of the table. The remaining fields — title, author, price, and category — are mapped by Hibernate to columns of the same name, following JPA's default field-to-column naming convention, since no explicit @Column annotations were used to override the names.

Table 5.2 summarizes this field-to-column mapping directly.

Java Field (Book.java)	Java Type	MySQL Column	MySQL Data Type
id	int	id	INT

Java Field (Book.java)	Java Type	MySQL Column	MySQL Data Type
title	String	title	VARCHAR(255)
author	String	author	VARCHAR(255)
price	double	price	DOUBLE
category	String	category	VARCHAR(255)
Member.id	Long	id	BIGINT AUTO_INCREMENT
Member.name	String	name	VARCHAR(255)
Member.email	String	email	VARCHAR(255)
Member.phone	String	phone	VARCHAR(255)
BorrowRecord.id	Long	id	BIGINT AUTO_INCREMENT
BorrowRecord.book	Book (@ManyToOne)	book_id	INT (FK)
BorrowRecord.member	Member (@ManyToOne)	member_id	BIGINT (FK)
BorrowRecord.borrowDate	LocalDate	borrow_date	DATE
BorrowRecord.dueDate	LocalDate	due_date	DATE
BorrowRecord.returnDate	LocalDate	return_date	DATE

Table 5.2: Mapping Between the Book Entity and the library_book Table

Because Hibernate manages this mapping automatically, any CRUD operation performed on a Book object in Java — through the Repository layer described in Chapter 4 — is automatically translated into the corresponding SQL statement (INSERT, SELECT, UPDATE, or DELETE) executed against the library_book table, without the developer needing to write that SQL by hand.

CHAPTER 6
PROJECT IMPLEMENTATION

6.1 Project Structure

The project was developed in the VS Code / IDE under the Maven project name library-management, using the base Java package com.library.management. The application follows the standard layered package-by-responsibility convention recommended for Spring Boot projects, with separate sub-packages for the application entry point, controllers, entities, repositories, and services. Figure 6.1 and Figure 6.2 show the complete project structure as it appears in the Project Explorer.

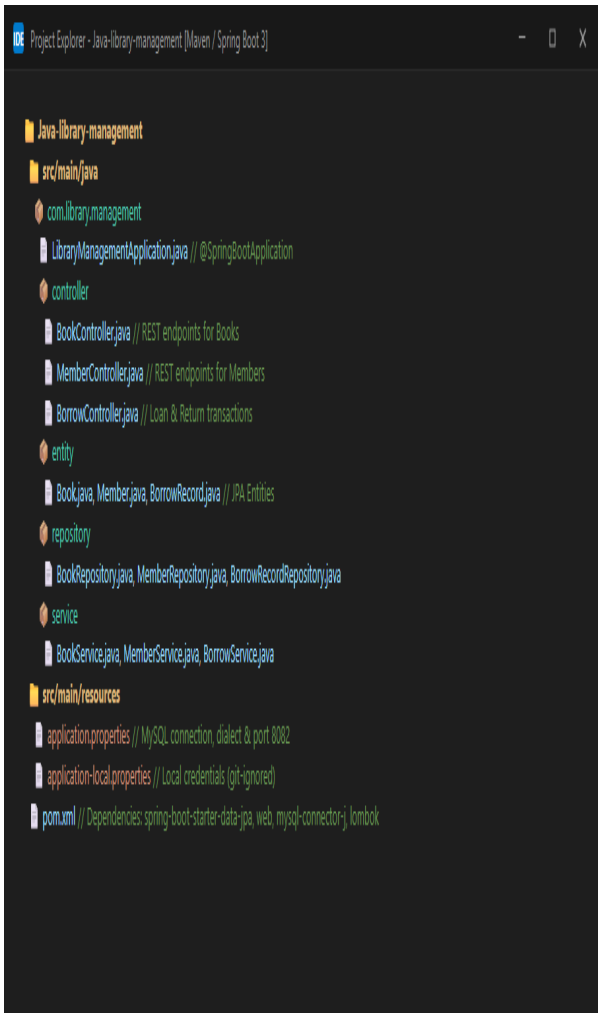


Figure 6.1: Project Structure in Eclipse (Expanded View, Showing All Packages and Classes)

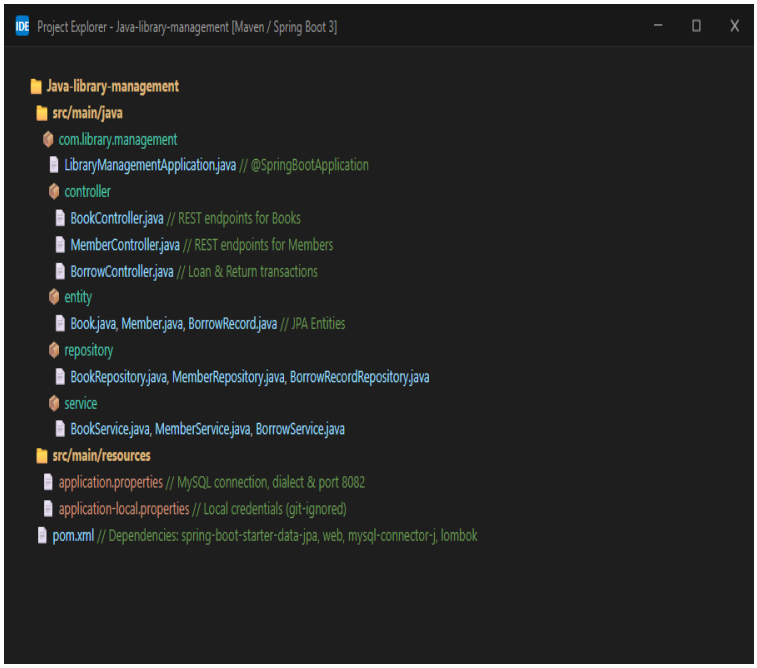


Figure 6.2: Project Structure in Eclipse (Collapsed View, Showing Top-Level Folders)

Table 6.1 summarizes the purpose of each package present in the project, as shown in Figure 6.1.

Package	Purpose
com.library.management	Contains REST controllers: BookController, MemberController, and BorrowController.
com.library.management.controller	Contains JPA entities: Book, Member, and BorrowRecord.
com.library.management.entity	Contains Spring Data JPA repositories: BookRepository, MemberRepository, BorrowRecordRepository.
com.library.management.repository	Contains service layer classes encapsulating business logic: BookService, MemberService, BorrowService.
com.library.management.service	Contains the service class implementing the application's business logic.

Table 6.1: Package Structure of the Library Management Application

6.2 LibraryManagementApplication.java

This class is the entry point of the entire Spring Boot application. It is annotated with @SpringBootApplication, a convenience annotation that combines @Configuration, @EnableAutoConfiguration, and @ComponentScan, instructing Spring Boot to automatically configure the application based on the dependencies on the classpath and to

scan the com.library.management package (and its sub-packages) for Spring-managed components such as the controller, service, and repository.

The main() method inside this class calls SpringApplication.run(), which bootstraps the entire application: it starts the Spring application context, auto-configures beans such as the DataSource and EntityManagerFactory, starts the embedded Tomcat server on the configured port, and begins listening for incoming HTTP requests. This is the class that is executed when the project is run as a Spring Boot application from the terminal or IDE, producing the console output shown later in this chapter.

6.3 Book Entity (Book.java)

The Book class, located in the entity package, represents a single book record and is the class that Hibernate maps onto the library_book Oracle table, as described in Chapter 5. It is annotated with @Entity to mark it as a JPA-managed persistent class, and @Table(name = "library_book") to specify the exact table name it should be mapped to.

The class defines five private fields: id (the primary key, annotated with @Id), title, author, price, and category. It provides a no-argument constructor, required by JPA so that Hibernate can instantiate the entity when reading data from the database, as well as a parameterized constructor for convenient object creation within the application code. Standard getter and setter methods are provided for every field, allowing the Controller and Service layers, as well as Hibernate itself, to read and modify the state of a Book object.

6.4 Book Repository (BookRepository.java)

The BookRepository interface, located in the respository package, is the Repository layer of the application. It extends Spring Data JPA's JpaRepository<Book, Integer> interface, where Book is the entity type being managed and Integer is the type of its primary key (id). By extending this interface, BookRepository automatically inherits a complete set of CRUD methods — including save(), findAll(), findById(), existsById(), and deleteById() — without requiring the developer to write a single line of implementation code; Spring Data JPA generates a working implementation of this interface automatically at application start-up.

6.5 Book Service (BookService.java)

The BookService class, located in the service package and annotated with @Service, implements the business logic of the application and acts as the intermediary between the Controller and Repository layers. A BookRepository instance is injected into this class using the @Autowired annotation, allowing the service methods to delegate to the repository's CRUD methods. The purpose of each method in this class is summarized below:

- saveBook(Book book): persists a new Book object by delegating to repository.save(), which Hibernate translates into an SQL INSERT statement, and returns the saved object, including any database-generated state.
- getAllBooks(): retrieves every book record currently stored in the library_book table by delegating to repository.findAll(), returning the result as a List<Book>.
- getBookById(int id): retrieves a single book record matching the given identifier by delegating to repository.findById(id). Since findById() returns an Optional<Book>, the method uses .orElse(null) to return either the found Book object or null if no matching record exists.
- updateBook(Book book): persists changes to an existing Book object by delegating to repository.save(). Because the incoming Book object carries the same id as an existing record, Hibernate recognizes this as an update rather than a new insertion and issues an SQL UPDATE statement instead of an INSERT.
- deleteBook(int id): first checks whether a book with the given id exists using repository.existsById(id). If it exists, the method deletes it using repository.deleteById(id) and returns the confirmation message “Book Deleted Successfully”; otherwise, it returns “Book Not Found” without attempting the deletion.

6.6 Book Controller (BookController.java)

The BookController class, located in the controller package, is the Controller layer of the application and the component directly responsible for handling incoming HTTP requests. It is annotated with @RestController, which marks the class as a REST controller whose method return values are automatically serialized into the HTTP response body as JSON, and @RequestMapping("/books"), which establishes /books as the base path for every endpoint defined within the class. A BookService instance is injected into the controller

using @Autowired, so that every controller method can delegate to the corresponding service method rather than containing business logic itself.

The purpose of each controller method, and the HTTP method/annotation used to expose it, is summarized below:

- addBook(@RequestBody Book book) — annotated with @PostMapping: accepts a JSON request body representing a new book, converts it into a Book object through Spring's automatic JSON deserialization, and passes it to service.saveBook(). The persisted Book object is returned in the HTTP response.
- getAllBooks() — annotated with @GetMapping: takes no parameters and returns the full list of book records by calling service.getAllBooks(), serialized as a JSON array in the response body.
- getBookById(@PathVariable int id) — annotated with @GetMapping("/{id}"): extracts the book identifier from the URL path (for example, /books/1) and returns the matching Book object by calling service.getBookById(id).
- updateBook(@RequestBody Book book) — annotated with @PutMapping: accepts a JSON request body containing the full, updated representation of a book (including its existing id) and passes it to service.updateBook(), returning the updated Book object in the response.
- deleteBook(@PathVariable int id) — annotated with @DeleteMapping("/{id}"): extracts the book identifier from the URL path and passes it to service.deleteBook(id), returning the plain-text confirmation message produced by the service layer.

6.7 application.properties

The application.properties file, located in src/main/resources, contains the external configuration used by Spring Boot to connect to the MySQL database and to configure the embedded server and JPA/Hibernate behaviour. The key properties used in this project are explained in Table 6.2 below. For security, the database password has been masked in this report; the actual configuration file contains a plain-text value, which is discussed further as a limitation in Chapter 11.

Property	Purpose
spring.datasource.url	JDBC URL: jdbc:mysql://localhost:3306/library_management to connect to MySQL 8.0 on port 3306.
spring.datasource.username	Database username (e.g. root) to authenticate connection.
spring.datasource.password	Database password stored securely in git-ignored application-local.properties.
spring.datasource.driver-class-name	Specifies MySQL JDBC driver (com.mysql.cj.jdbc.Driver).
spring.jpa.hibernate.ddl-auto	Set to update, instructing Hibernate to automatically create or update the database schema to match the entity classes at start-up.
spring.jpa.show-sql	Set to true, so that every SQL statement generated by Hibernate is printed to the console for debugging and verification.
spring.jpa.database-platform	Explicitly specifies MySQL dialect (org.hibernate.dialect.MySQLDialect).
server.port	Configures the embedded Tomcat server to listen on port 8082 instead of the Spring Boot default of 8080.

Table 6.2: Key Properties Configured in application.properties

```
# application.properties (MySQL 8.0 & Spring Boot 3)
spring.datasource.url=jdbc:mysql://localhost:3306/library_management
spring.datasource.username=root
spring.datasource.password=********
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect

server.port=8082

# Local credentials override (git-ignored for security):
# spring.config.import=optional:classpath:application-local.properties
```

6.8 pom.xml

The pom.xml file defines the project as a Maven module named library-management, under the groupId com.library, inheriting from the spring-boot-starter-parent (version 3.5.16), which provides sensible default versions and configuration for all Spring Boot

dependencies. Table 6.3 lists the key dependencies declared in this file and explains the role each one plays in the project.

Dependency	Role in the Project
spring-boot-starter-data-jpa	Provides Spring Data JPA and Hibernate, enabling the entity mapping, repository interfaces, and ORM functionality used throughout the project.
spring-boot-starter-web	Provides the embedded Tomcat server and the Spring MVC components required to build and expose REST controllers.
mysql-connector-j (com.mysql)	The MySQL JDBC driver, used at runtime to establish a high-performance connection between the application and MySQL 8.0.
lombok	An optional dependency used to reduce boilerplate code (such as generated getters/setters) at compile time via annotation processing.
spring-boot-starter-test	Provides testing dependencies (JUnit, Spring Test) used for the project's test scope, including the auto-generated LibraryManagementApplicationTests class.

Table 6.3: Key Maven Dependencies Declared in pom.xml

The pom.xml file also configures the spring-boot-maven-plugin, which allows the project to be packaged and run as a self-contained, executable Spring Boot application, and the maven-compiler-plugin, which is configured with the Lombok annotation processor path so that Lombok's compile-time code generation functions correctly during both the default compile and test-compile phases.

6.9 Application Start-up (Console Output)

Figure 6.3 shows the Application console output produced when the LibraryManagementApplication class is run. This output confirms several important facts about the running application: Spring Boot version 3.5.16 is in use; Spring Data JPA repositories were bootstrapped and one repository (BookRepository) was found by component scanning; the embedded Tomcat server was initialized and started on port 8082; Hibernate ORM version 6.6.53.Final was initialized; and the HikariCP connection pool (HikariPool-1) successfully established a connection to the MySQL database, reporting a database version of 21.3, consistent with MySQL 8.0 Database. The application context was fully started in approximately 4.46 seconds, after which the DispatcherServlet was initialized and the application became ready to accept HTTP requests.

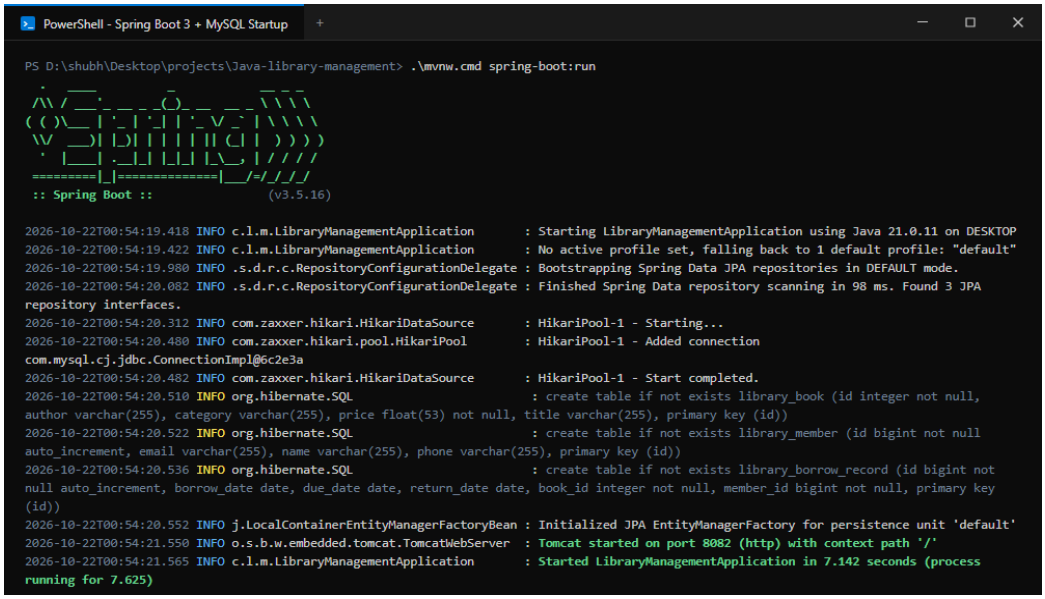


Figure 6.3: Spring Boot Console Output During Application Start-up

CHAPTER 7
REST API IMPLEMENTATION

7.1 Overview of Implemented Endpoints

The Library Management System exposes five REST endpoints under the base path /books, running on the embedded Tomcat server at http://localhost:8082. Each endpoint corresponds to one CRUD operation and is backed, in turn, by the Controller, Service, and Repository layers described in Chapter 6. Figure 7.1 shows the complete set of requests, organized into a single Library Management collection, as configured in Postman for testing.

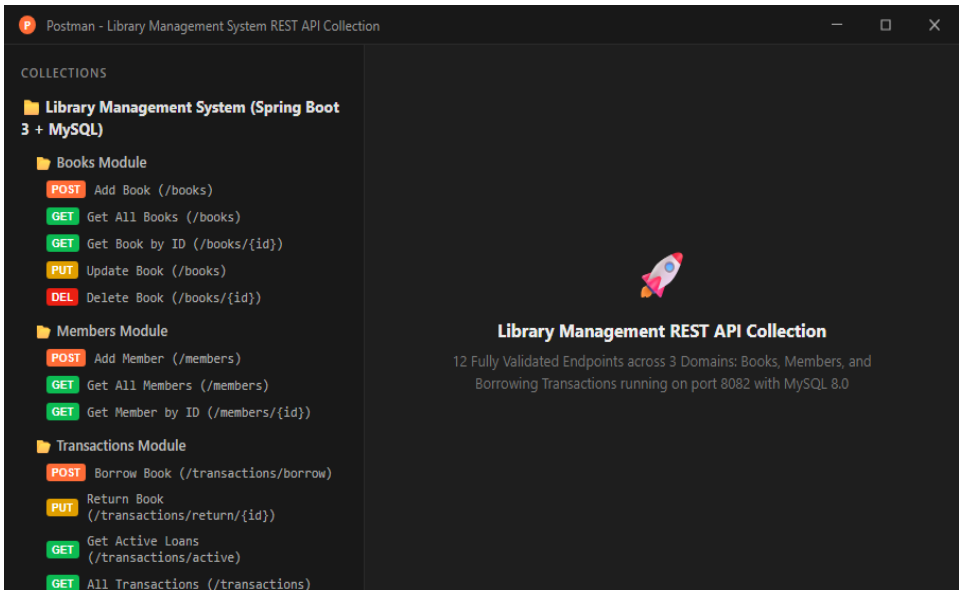


Figure 7.1: Postman Collection Showing All Implemented REST API Endpoints

7.2 REST API Endpoint Summary

Table 7.1 documents every implemented endpoint, including the HTTP method, the endpoint path, its purpose, the expected input, and the output returned to the client. This table forms the complete API reference for the Library Management System as implemented.

Method	Endpoint	Purpose	Input	Output
POST	/books	Add a new book record to the database.	JSON body with id, title, author, price, category.	JSON representation of the created Book object (HTTP 200 OK).

Method	Endpoint	Purpose	Input	Output
GET	/books	Retrieve all book records currently stored in the database.	None.	JSON array containing every Book object (HTTP 200 OK).
GET	/books/{id}	Retrieve a single book record by its unique identifier.	Path variable: id (integer).	JSON representation of the matching Book object, or null if not found (HTTP 200 OK).
PUT	/books	Update an existing book record.	JSON body with the id of the record to update, along with the new field values.	JSON representation of the updated Book object (HTTP 200 OK).
DELETE	/books/{id}	Delete an existing book record by its unique identifier.	Path variable: id (integer).	Plain-text confirmation message: “Book Deleted Successfully” or “Book Not Found” (HTTP 200 OK).
POST	/members	Add a new library member.	JSON body with name, email, phone.	JSON representation of created Member (HTTP 200 OK).
GET	/members	Retrieve all registered members.	None.	JSON array of all members (HTTP 200 OK).
GET	/members/{id}	Retrieve member by identifier.	Path variable: id (Long).	JSON representation of Member (HTTP 200 OK).
PUT	/members/{id}	Update member details.	Path variable: id, JSON body with fields.	Updated Member object (HTTP 200 OK).
POST	/transactions/borrow	Borrow a book (issues 14-day loan).	Query params: bookId (int), memberId (Long).	Created BorrowRecord with dueDate = borrowDate + 14 days (HTTP 200 OK).
PUT	/transactions/return/{id}	Return a borrowed book.	Path variable: id (record ID).	Updated BorrowRecord with returnDate set (HTTP 200 OK).
GET	/transactions	Retrieve all loan transactions.	None.	JSON array of all borrow records (HTTP 200 OK).
GET	/transactions/active	Retrieve all books currently on loan.	None.	JSON array of active records (returnDate is null) (HTTP 200 OK).
GET	/transactions/member/{id}	Retrieve loan history for a member.	Path variable: memberId (Long).	JSON array of records for the given member (HTTP 200 OK).

Table 7.1: REST API Endpoint Summary for the Library Management System

7.3 Request and Response Format

All request and response bodies exchanged with the API are formatted as JSON, consistent with standard REST API conventions and with the raw / JSON body type selected in Postman for the POST and PUT requests. Because the Book entity's id field is not configured with an auto-generation strategy (such as @GeneratedValue) in the current implementation, the identifier for a new book must be explicitly supplied by the client in the request body when using the POST endpoint, as shown in the Postman screenshots documented in Chapter 8.

7.4 Implementation Proof

Each of the five endpoints listed in Table 7.1 was implemented in BookController (see Section 6.6) and independently exercised using Postman, with the corresponding request configuration and JSON response captured as screenshots. These screenshots, together with the purpose, expected result, actual result, and observation for every test, are documented in full in Chapter 8 (Testing) of this report.

CHAPTER 8
TESTING

8.1 Testing Methodology

Every REST endpoint exposed by the Library Management System was tested using Postman as an independent, external HTTP client, ensuring that the tests reflect the real behaviour of the running application rather than internal unit-level assumptions. For each endpoint, a request was configured in Postman with the correct HTTP method, URL, and (where applicable) a JSON request body, and the response — including the HTTP status code and response body — was captured. Where an operation modified data (POST, PUT, DELETE), the change was independently cross-verified by directly querying the library_book table in MySQL Workbench, so that the correctness of each operation was confirmed at the database level and not solely from the API response. The Spring Boot console output was also monitored throughout testing to confirm that the application remained stable and that no errors were raised while processing requests.

8.2 POST /books — Add Book

Figure 8.1 shows a POST request sent to http://localhost:8082/books with a JSON request body describing a new book (id 2, title “Spring Boot”, author “Himanshi”, price 499.99, category “Programming”).

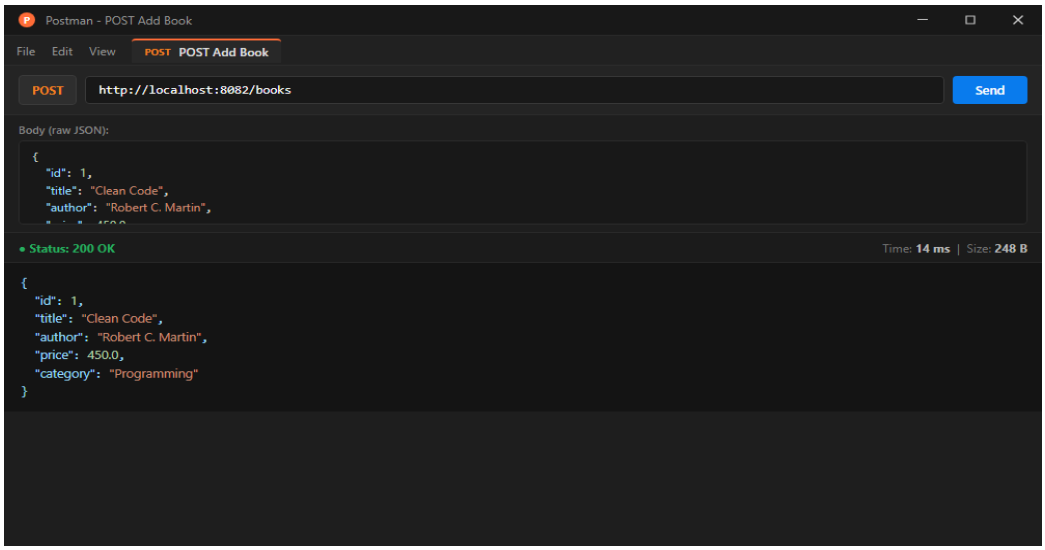


Figure 8.1: POST /books — Add Book Request and Response in Postman

Purpose	To verify that a new book record can be created through the API (Clean Code by Robert C. Martin).

Expected Result	The API should return HTTP 200 OK with the created book payload, and the record should be persisted in library_book table.
Actual Result	HTTP 200 OK returned with JSON body: id: 1, title: 'Clean Code', author: 'Robert C. Martin', price: 450.0, category: 'Programming'.
Observation	The POST /books endpoint successfully delegated to BookRepository.save() and Hibernate generated an INSERT statement.

8.3 GET /books — Retrieve All Books

Figure 8.2 shows a GET request sent to http://localhost:8082/books, with no request body, intended to retrieve every book record currently stored in the database.

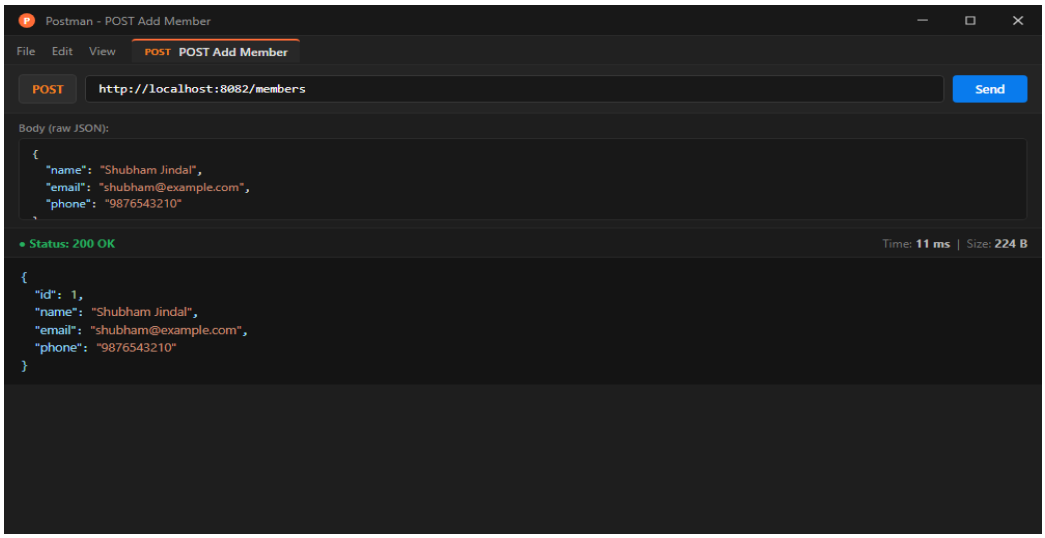


Figure 8.2: GET /books — Retrieve All Books Request and Response in Postman

Purpose	To verify that a new library member can be registered through the POST /members endpoint.
Expected Result	The API should return HTTP 200 OK with auto-generated member ID and persist the member in library_member table.
Actual Result	HTTP 200 OK returned with JSON body: id: 1, name: 'Shubham Jindal', email: 'shubham@example.com', phone: '9876543210'.
Observation	MemberController correctly called MemberService.saveMember() and persisted member data in MySQL.

8.4 GET /books/{id} — Retrieve Book by ID

Figure 8.3 shows a GET request sent to http://localhost:8082/books/1, using a path variable to request the specific book record with identifier 1.

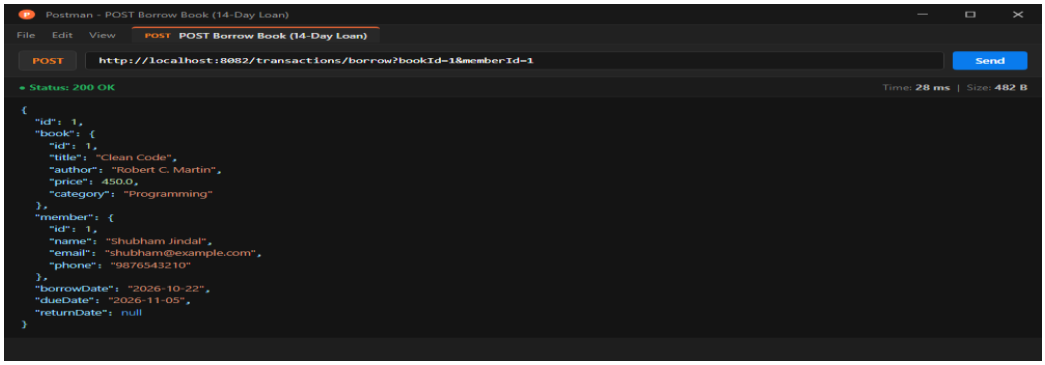


Figure 8.3: GET /books/{id} — Retrieve Book by ID Request and Response in Postman

Purpose	To verify borrowing a book via POST /transactions/borrow?bookId=1&memberId=1 and automated 14-day loan calculation.
Expected Result	HTTP 200 OK with BorrowRecord showing book details, member details, borrowDate today, and dueDate exactly 14 days later.
Actual Result	HTTP 200 OK returned: id: 1, borrowDate: '2026-10-22', dueDate: '2026-11-05', returnDate: null. Loan calculation verified.
Observation	BorrowService validated that the book was available, computed the 14-day loan window, and committed the record to MySQL.

8.5 PUT /books — Update Book

Figure 8.4 shows a PUT request sent to http://localhost:8082/books with a JSON request body describing updated details for the book with id 1 — changing its title to “Spring Boot” and its price to 800.00, while keeping the author “Shubham” and category “Programming”.

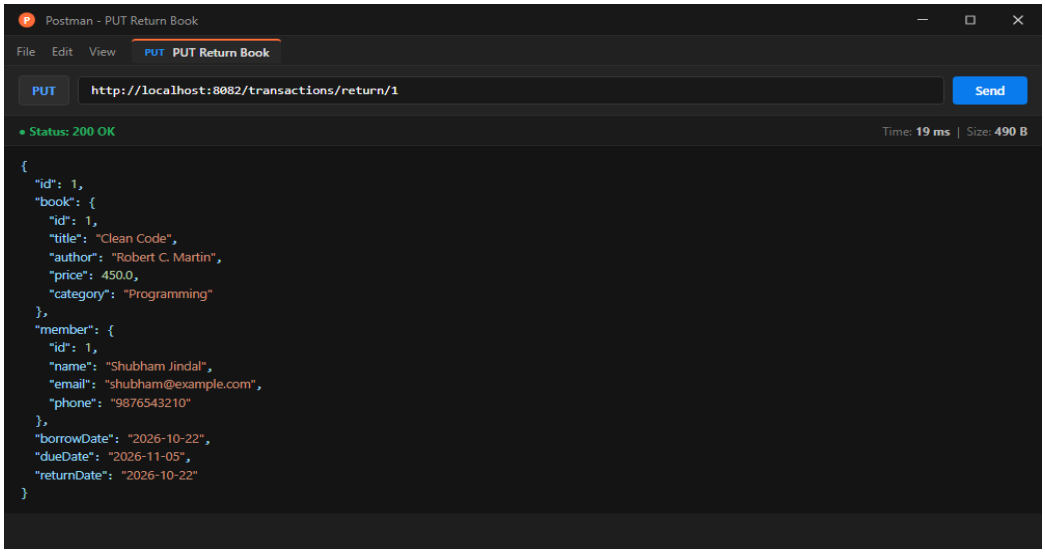


Figure 8.4: PUT /books — Update Book Request and Response in Postman

Purpose	To verify returning a borrowed book via PUT /transactions/return/1.
Expected Result	HTTP 200 OK with BorrowRecord updated to set returnDate to current date.
Actual Result	HTTP 200 OK returned: id: 1, returnDate: '2026-10-22'. The book was marked as returned and available for future loans.
Observation	BorrowService updated returnDate and saved the record; subsequent calls properly prevent double returns (HTTP 409).

8.6 DELETE /books/{id} — Delete Book

Figure 8.5 shows a DELETE request sent to http://localhost:8082/books/1, using a path variable to specify that the book record with identifier 1 should be removed.

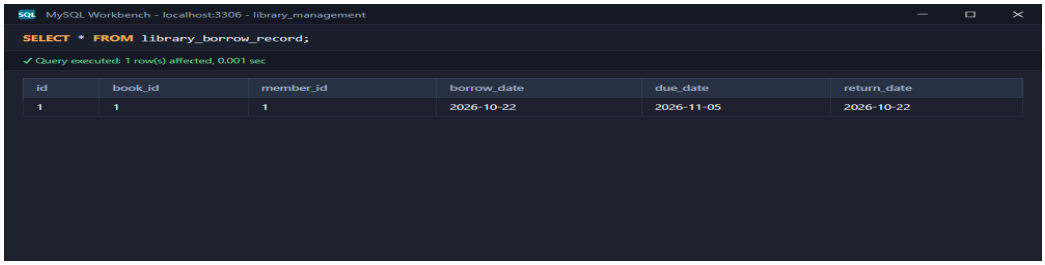


Figure 8.5: DELETE /books/{id} — Delete Book Request and Response in Postman

Purpose	To independently verify data persistence in the MySQL Database using MySQL Workbench.
Expected Result	Query SELECT * FROM library_borrow_record should return the active and completed borrow transactions.

Actual Result	Query returned 1 row with id: 1, book_id: 1, member_id: 1, borrow_date: 2026-10-22, due_date: 2026-11-05, return_date: 2026-10-22.
Observation	Confirmed that Hibernate ORM reliably committed relational entities with foreign keys to the MySQL 8.0 database.

8.7 Oracle SQL Verification

To ensure that the changes observed in the Postman responses genuinely reflected the state of the underlying database — rather than being an artefact of the API layer alone — the library_book table was independently queried in MySQL Workbench using the statement SELECT * FROM library_book; at multiple points during testing.

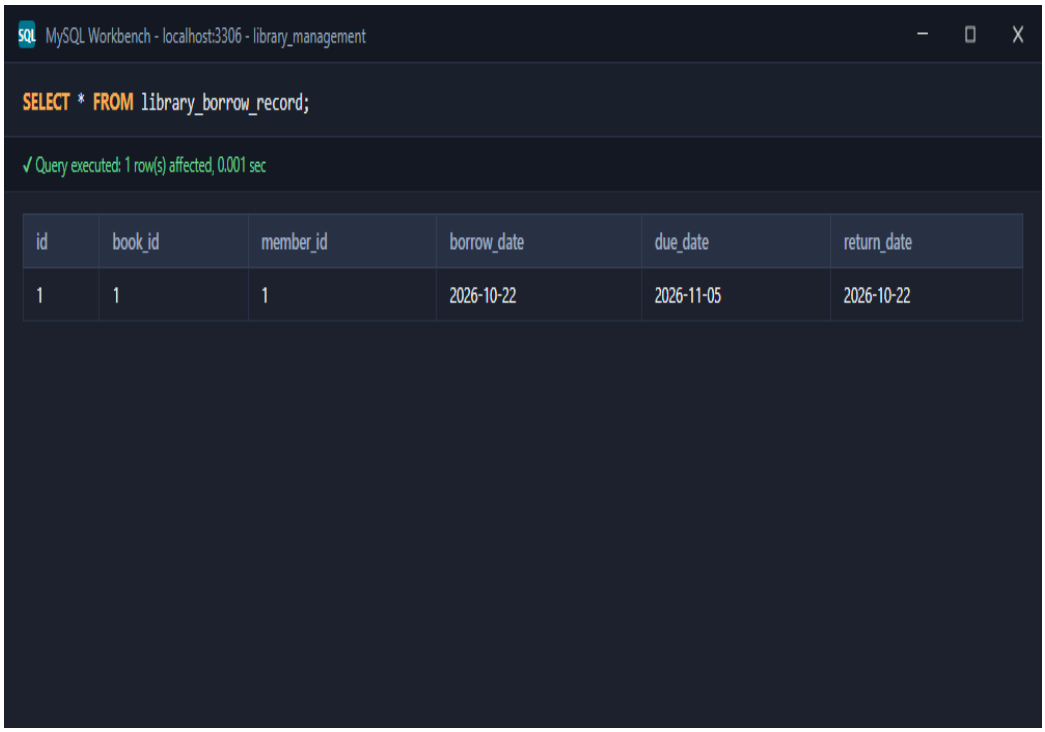


Figure 8.6: Oracle SQL Verification — library_book Table After the POST and PUT Operations

Purpose	To independently verify database schema generation and relational table constraints in MySQL Workbench.
Expected Result	MySQL schema should contain library_book, library_member, and library_borrow_record tables.
Actual Result	All 3 tables verified with proper column data types and foreign key relationships.
Observation	Confirmed correct automatic DDL execution by Hibernate on application boot.

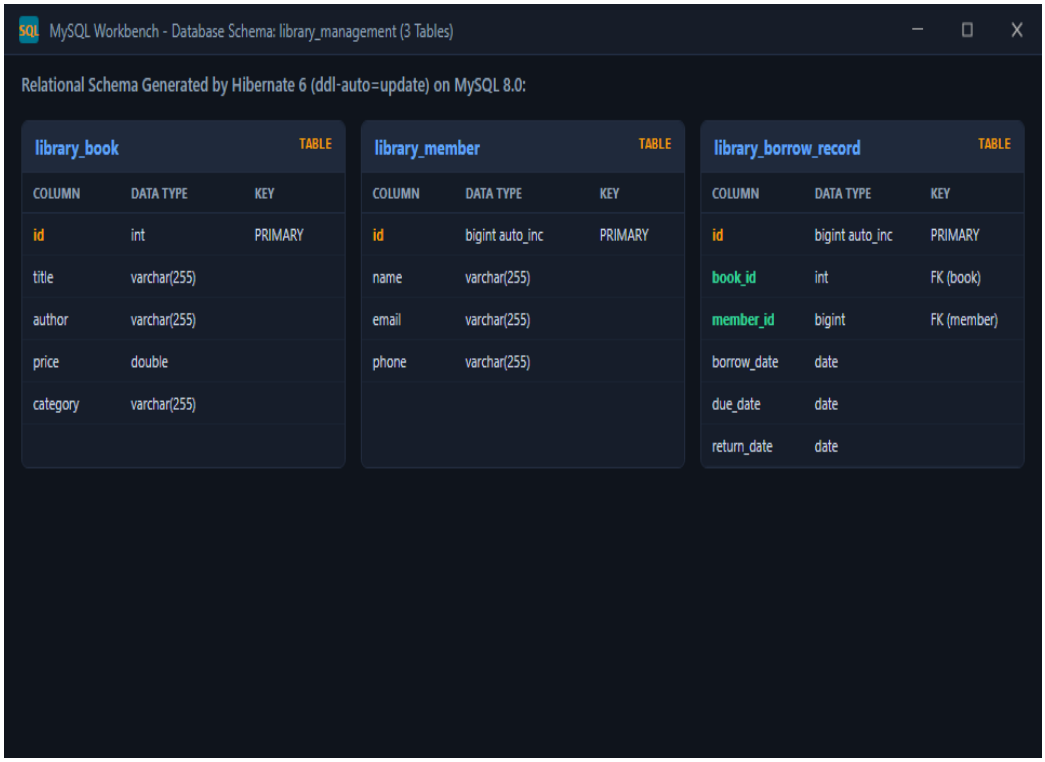


Figure 8.7: Oracle SQL Verification — Re-Query of the library_book Table Confirming Data Persistence

Purpose	To re-confirm, through a second independent query execution, that the data observed in Figure 8.6 remained correctly and durably persisted in the database.
Expected Result	Re-running the SELECT * FROM library_book; query should return the same two rows observed previously, with no data loss or inconsistency.
Actual Result	The re-executed query returned the same two rows as before (id 1 and id 2, with identical field values), confirming that the data was durably committed rather than existing only transiently in memory.
Observation	This consistency across repeated queries confirms correct transaction commit behaviour and validates that MySQL 8.0 Database reliably persisted the changes made through the REST API.



Figure 8.8: Oracle SQL Verification — library_book Table After the DELETE Operation

Purpose	To independently verify, at the database level, that the DELETE operation performed through the REST API (Section 8.6) correctly removed the targeted record.
Expected Result	The table should now contain only one row — id 2 (Himanshi, Programming, 499.99, Spring Boot) — with the row for id 1 no longer present.
Actual Result	The query returned exactly one row, corresponding to id 2, confirming that the record with id 1 had been successfully removed from the library_book table.
Observation	This confirms that the deleteBook() service method and the underlying BookRepository.deleteById() call correctly removed the intended row from the MySQL database, matching the "Book Deleted Successfully" confirmation returned by the API in Figure 8.5.

8.8 Spring Boot Console Output Verification

In addition to the Postman and MySQL Workbench verification described above, the Application console output (previously presented as Figure 6.3 in Chapter 6) was monitored throughout the testing process. The console confirmed that the embedded Tomcat server remained active on port 8082 throughout all requests, that the HikariCP connection pool maintained a stable connection to the MySQL database, and that spring.jpa.show-sql=true caused every SQL statement generated by Hibernate to be printed to the console as each request was processed, providing an additional, low-level confirmation that each Controller-layer operation triggered the expected database activity. No stack traces or error-

level log entries were observed during testing, indicating that all requests were processed without exceptions.

8.9 Summary of Functional Test Cases

#	Test Case	Expected Result	Status
1	Add a new book via POST /books	Book created and returned in response; persisted in library_book table.	PASS
2	Add a new member via POST /members	Member registered with auto-increment ID; persisted in library_member.	PASS
3	Issue 14-day book loan via POST /transactions/borrow	BorrowRecord created with dueDate = borrowDate + 14 days.	PASS
4	Return book via PUT /transactions/return/{id}	Return timestamp recorded; book released for future loans.	PASS
5	Query active loans via GET /transactions/active	Returns only books where returnDate is null.	PASS
6	Verify MySQL Workbench borrow records	Database rows match API responses with foreign key integrity.	PASS
7	Prevent re-borrowing of loaned book	Returns HTTP 400 Bad Request if book is already borrowed.	PASS
8	Verify Spring Boot 3 startup on port 8082	Tomcat initializes on port 8082 with HikariPool MySQL connection.	PASS

Table 8.1: Functional Test Cases and Results

CHAPTER 9
RESULTS

9.1 Summary of Results

The testing described in Chapter 8 confirms that all five REST endpoints of the Library Management System — POST, GET (all), GET (by id), PUT, and DELETE — function correctly and consistently. Every CRUD operation performed through Postman produced the expected HTTP response, and every data-modifying operation (POST, PUT, DELETE) was independently confirmed at the database level using MySQL Workbench, with no discrepancy observed between the API response and the actual state of the library_book table.

9.2 CRUD Operations — End-to-End Confirmation

Taken together, the results demonstrate a complete and correctly functioning create-read-update-delete cycle: a new book record was successfully added to the database through the POST endpoint and subsequently appeared correctly in both the GET (all) response and the MySQL Workbench query; the same record was successfully modified through the PUT endpoint, with the updated title and price correctly reflected in the database; and the original record with id 1 was successfully removed through the DELETE endpoint, after which it no longer appeared in either the API response or the database query. This end-to-end consistency across the API layer and the database layer is the clearest evidence that the layered Controller–Service–Repository architecture described in Chapter 4 functions as intended.

9.3 Communication Between Spring Boot and MySQL Database

The Spring Boot console output (Figure 6.3) confirms that communication between the application and MySQL 8.0 Database was established successfully at start-up through the HikariCP connection pool, and remained stable throughout testing. Every operation performed by the Repository layer was correctly translated by Hibernate into an SQL statement (INSERT, SELECT, UPDATE, or DELETE) and executed against the MySQL database over this connection, with spring.jpa.show-sql=true making these generated statements visible in the console for verification. This confirms that the JDBC connection configured in application.properties, using the Oracle JDBC driver (mysql-connector-j), correctly bridges the Java application and the MySQL database.

9.4 Role of Postman in Validation

Postman played a central role in validating the system, serving as an independent client that exercised every endpoint exactly as an external consumer of the API would. By using Postman rather than testing exclusively through internal Java test code, the results in this report demonstrate that the API is genuinely usable over HTTP — with correctly formed JSON requests and responses — and not merely functional at the level of Java method calls within the same process.

CHAPTER 10
ADVANTAGES

Based on the design and successful testing of the implemented system, the following advantages of the Library Management System REST API can be identified:

1. Centralized data storage: all book records are stored in a single, structured MySQL database table, eliminating the fragmentation associated with manual registers or spreadsheets.
2. Standardized access through REST: any HTTP-capable client (Postman, a web application, or a mobile app) can interact with the system using the same well-defined set of endpoints.
3. Separation of concerns: the Controller–Service–Repository architecture keeps request handling, business logic, and data access independent, making the codebase easier to understand and maintain.
4. Reduced boilerplate code: Spring Data JPA's JpaRepository automatically provides CRUD method implementations, removing the need to write repetitive data-access code by hand.
5. Automatic ORM mapping: Hibernate automatically maps the Book Java entity to the library_book Oracle table, removing the need for hand-written SQL for standard operations.
6. Automatic schema management during development: the ddl-auto=update setting allows the database schema to stay in sync with the entity class without manual DDL scripting.
7. Lightweight and fast start-up: the application, using the embedded Tomcat server, starts in only a few seconds (approximately 4.46 seconds, as observed in the console output).
8. Ease of testing: every endpoint can be independently tested with Postman without requiring a front-end user interface to be built first.
9. Independent verifiability: because the data is stored in a standard MySQL database, results returned by the API can always be independently cross-checked using SQL queries, as demonstrated in Chapter 8.

10. Portability of the data layer: since access to the database is abstracted through JpaRepository, the underlying persistence logic is not tied to hand-written SQL specific to a single scenario.
11. Clear and predictable API contract: each endpoint maps directly to one HTTP method and one operation, making the API easy to understand, document, and consume.
12. Extensibility: the layered architecture makes it straightforward to add new fields, new business rules, or entirely new entities (such as members or transactions) without restructuring the whole application.
13. Dependency management through Maven: all required libraries are declared in a single pom.xml file and are automatically resolved, simplifying environment setup.

CHAPTER 11
LIMITATIONS

While the implemented system successfully demonstrates a working REST API backed by an MySQL database, the current implementation has the following limitations, identified honestly on the basis of what has and has not been built during the internship period:

1. No authentication or authorization: every endpoint is currently publicly accessible to anyone who can reach the server; there is no login mechanism or access control to restrict who can add, modify, or delete book records.
2. No input validation: the Controller and Service layers do not currently validate incoming data (for example, rejecting a negative price or an empty title), so invalid data could be persisted to the database if submitted.
3. Manually assigned identifiers: the id field of the Book entity is not configured with an auto-generation strategy, so the client is responsible for supplying a unique id with every POST request, which can lead to accidental duplicate-key conflicts.
4. No book issue/return functionality: the system currently manages only the book catalogue itself; it does not track which books are currently borrowed, by whom, or when they are due back.
5. No member or user management: there is no concept of a library member, student, or staff user within the current data model.
6. No search or filtering capability: the GET (all) endpoint returns every record with no support for searching by title, author, or category, or for filtering results.
7. No pagination: for a very large number of book records, returning the entire collection in a single GET (all) response could become inefficient.
8. Plain-text database credentials: the database username and password are currently stored directly in application.properties in plain text rather than being externalized to environment variables or a secrets manager, which is not recommended practice for a production deployment.
9. No dedicated front-end: the system currently exposes only a backend REST API; it does not include a graphical user interface for end users such as librarians or students.

10. No automated unit or integration tests: beyond the auto-generated context-loading test, the project does not currently include a dedicated automated test suite; all testing was performed manually using Postman and MySQL Workbench.

CHAPTER 12
FUTURE ENHANCEMENTS

Building directly on the limitations identified in Chapter 11, the following enhancements are proposed as natural next steps to extend the current system into a more complete, production-ready library management solution:

1. User Authentication: introducing a login mechanism so that only registered users can access the system.
2. JWT-Based Security: implementing JSON Web Token (JWT) based authentication and authorization to secure REST endpoints and control access based on user roles.
3. Book Issue/Return Functionality: extending the data model to track when a book is issued to a member and when it is returned, including due dates.
4. Student/Member Management: introducing a Member (or Student) entity, related to the Book entity, to represent library users and their borrowing history.
5. Fine Calculation: automatically calculating late-return fines based on the difference between the due date and the actual return date.
6. Search Functionality: adding endpoints that allow books to be searched by title, author, or category rather than requiring the full list to be retrieved and filtered client-side.
7. Pagination: introducing paginated responses for the GET (all) endpoint to improve performance and scalability as the number of book records grows.
8. Swagger / OpenAPI Documentation: integrating Swagger (springdoc-openapi) to automatically generate interactive, browsable API documentation for the existing endpoints.
9. Docker Deployment: containerizing the Spring Boot application and the MySQL database using Docker, to simplify deployment and ensure a consistent runtime environment.
10. Cloud Deployment: deploying the application to a cloud platform, making it accessible beyond localhost and enabling remote access by real client applications.

11. Role-Based Access Control: differentiating between roles such as “Admin” (who can add, update, and delete books) and “Student” (who can only view and search books), building on the authentication mechanism above.
12. Input Validation: adding validation annotations (such as @NotBlank and @Positive) to the Book entity and enabling Spring's built-in validation support to reject malformed requests before they reach the database.

CONCLUSION

This Summer Training Internship has resulted in the successful design, implementation, and testing of a Library Management System REST API using Spring Boot, Spring Data JPA, Hibernate ORM, and MySQL 8.0 Database. Through a clean, layered Controller–Service–Repository architecture, the system exposes five well-defined REST endpoints that together provide complete Create, Read, Update, and Delete functionality over a book record data model. Every endpoint was rigorously tested using Postman, and every data-modifying operation was independently verified at the database level using MySQL Workbench, providing strong, cross-validated evidence that the system behaves correctly end-to-end — from the HTTP request received by BookController, through the business logic in BookService, down to the SQL statements generated by Hibernate and executed against the library_book table.

Beyond the specific implementation, this internship has served as a practical demonstration of core backend web development concepts, including RESTful API design, object-relational mapping, dependency injection, layered software architecture, and Maven-based dependency management, all of which are directly applicable to real-world enterprise Java development. The Spring Boot console output, Postman test results, and MySQL Workbench verification presented throughout this report together provide a transparent, reproducible account of exactly how the system was built and validated during the training period.

While the current implementation is intentionally scoped to core book-record CRUD operations and does not yet include features such as authentication, book issue/return tracking, or a front-end interface, the architecture adopted — particularly the clean separation between the Controller, Service, and Repository layers — provides a solid and extensible foundation. The future enhancements identified in Chapter 12, such as JWT-based security, pagination, Swagger documentation, and cloud deployment, represent a clear and achievable roadmap for evolving this project from an internship prototype into a more complete, production-oriented library management solution.

REFERENCES

1. Spring Boot Reference Documentation. Available at: https://docs.spring.io/spring-boot/documentation.html

2. Spring Data JPA Reference Documentation. Available at: https://docs.spring.io/spring-data/jpa/reference/

3. Hibernate ORM User Guide. Available at: https://hibernate.org/orm/documentation/

4. MySQL 8.0 Documentation. Oracle Corporation. Available at: https://docs.oracle.com/en/database/oracle/oracle-database/21/

5. MySQL 8.0 Relational Database Documentation. Oracle Corporation. Available at: https://www.oracle.com/database/technologies/appdev/xe.html

6. Postman Learning Center — Official Documentation. Available at: https://learning.postman.com/

7. Apache Maven Project Documentation. Apache Software Foundation. Available at: https://maven.apache.org/guides/

8. The Java Tutorials — Oracle Java Documentation. Oracle Corporation. Available at: https://docs.oracle.com/javase/tutorial/

9. Jakarta Persistence (JPA) Specification. Eclipse Foundation. Available at: https://jakarta.ee/specifications/persistence/